COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

09

Gaming Platforms and Hardware/Software Co-design

Student handbook · Expanded Volume 08

PACKAGE 09 / VOLUME 08 / SEPTEMBER 28, 2026
Modules 36–40 · Gate H · Expanded teaching edition 1.0
Study, implement, test and review. Models are not physical measurements.

Contents

How to use this teaching volume 3
Program map 3
Module 36 — Embedded platforms, drivers and board design 4
Lesson 36.1 — Serial interfaces and peripheral contracts 5
Lesson 36.2 — Firmware, RTOS scheduling and driver lifecycle 7
Lesson 36.3 — Schematics, PCB layout, power integrity and signal integrity 9
Lesson 36.4 — Bring-up, instrumentation and fault isolation 11
Module 37 — GPU microarchitecture and memory systems 13
Lesson 37.1 — SIMT execution, divergence, occupancy and latency hiding 14
Lesson 37.2 — Coalescing, local storage, caches and shared-memory banks 16
Lesson 37.3 — Raster, texture, matrix and ray-tracing units 18
Lesson 37.4 — CPU-GPU sharing, discrete memory and synchronization 20
Module 38 — Quantitative performance engineering 22
Lesson 38.1 — Amdahl, roofline and bottleneck models 23
Lesson 38.2 — Profilers, counters and trustworthy benchmark design 25
Lesson 38.3 — SIMD, cache blocking, NUMA and job scheduling 27
Lesson 38.4 — Energy, thermal behavior and sustained performance 29
Module 39 — Gaming-platform constraints and interactive latency 31
Lesson 39.1 — Input-to-display latency and frame pacing 32
Lesson 39.2 — Mobile and handheld systems: memory, power and lifecycle 34
Lesson 39.3 — XR spaces, stereo views, pose timing and comfort constraints 36
Lesson 39.4 — Portability, platform services and console-development boundaries 38
Module 40 — Hardware/software co-design capstone 40
Lesson 40.1 — Workload characterization and measurable requirements 41
Lesson 40.2 — Architecture alternatives, arithmetic precision and dataflow 43
Lesson 40.3 — Interface design, virtualization and end-to-end evaluation 45
Lesson 40.4 — Design review, evidence tiers and undergraduate CE completion 47
Sources and provenance 49

How to use this teaching volume

Package 09 is the expanded teaching edition of Volume 08 in Computer Science and Computer Engineering for Games. It connects embedded interfaces, GPU execution, quantitative performance, platform constraints and hardware/software co-design. The through-line is a trustworthy engineering claim: state the useful result, define the boundary, implement a contract, test what can fail and distinguish measurements from assumptions.

There are 20 lessons, each followed by five quiz questions. Questions 1–3 retain the original curriculum; questions 4–5 add application work. Use LAB_WORKBOOK alongside the handbook for the full practical assignments, starter contracts, written problems and module projects. Keep ANSWER_KEY closed until after an attempt. COURSE.html provides the same teaching material with answer guidance collapsed by default.

The Python computational slices make one mechanism testable per lesson. They are not substitutes for complete driver engineering, a KiCad schematic/layout, native performance investigation or the learner's Gate H portfolio. Each workbook entry states the broader original assignment and required evidence. The assigned R-code study remains part of completion; the source directory resolves those references and adds checked official S-code resources.

The original planning range is 350–560 hours, including readings, practice, projects and revision. This book is a guided teaching volume, not an accredited degree or a promise that reading alone confers professional engineering competence. Follow START_NEXT for setup, readiness and the first session.

Program map

Module	Focus	Project evidence
36	Embedded platforms, drivers and boards	Simulated peripheral plus reviewable schematic/layout
37	GPU architecture and memory	Execution/data-movement workbook
38	Quantitative performance	Controlled investigation with raw data and uncertainty
39	Platform constraints and latency	Strict backend, lifecycle, memory and timing evidence
40	Hardware/software co-design	Gate H: workload-centered architecture decision

All required computational examples are software models. Physical work is optional and reviewed. The optional C++ reference benchmark measures host code, not modeled hardware. Model clocks, rates and resource counts remain assumptions unless supported by a separately labeled evidence tier.

Module 36 — Embedded platforms, drivers and board design

Prerequisites: Modules 17-18 and 31-35. Physical work remains optional and supervised.

Original module project: Design a simulated controller or peripheral with firmware, a host interface and a schematic-level review. A physical board extension begins only after the electrical, layout and bring-up plans receive qualified review.

Lesson	Learning outcome
36.1 · Serial interfaces and peripheral contracts	Choose a communication interface using timing and electrical constraints.
36.2 · Firmware, RTOS scheduling and driver lifecycle	Bound embedded work and make device state transitions explicit.
36.3 · Schematics, PCB layout, power integrity and signal integrity	Translate a circuit into a manufacturable layout without ignoring physics.
36.4 · Bring-up, instrumentation and fault isolation	Turn an unresponsive device into a sequence of testable hypotheses.

Read the module-project brief in the workbook before beginning. The four lessons build evidence for that project rather than four unrelated demonstrations.

Lesson 36.1 — Serial interfaces and peripheral contracts

Learning outcome: Choose a communication interface using timing and electrical constraints.

A peripheral is a contract, not a cable

A controller reading a thumbstick is a small distributed system. The host and peripheral have separate state, different clocks, and opportunities to reset independently. An electrically valid waveform may still express an invalid command. Separate the electrical contract (voltage domains, pin directions, rise times and loading), transport contract (clock, framing and bit order), and application contract (command meaning, sequence numbers, error policy and ownership). Debugging becomes difficult when these layers are blended into one phrase such as “SPI works.”

SPI commonly uses a controller-generated clock and device selection; the selected clock polarity and phase determine which edges carry and sample data. Zephyr documents controller/peripheral roles and device-relative data directions. Match the installed API and peripheral datasheet rather than copying signal labels from an unrelated board. UART and I2C require different timing and electrical reasoning; a bus name alone never establishes voltage compatibility. [S01; R53]

Write the table before writing the driver

For the educational controller, define a five-byte reply: sync `0xA5`, sequence, high payload byte, low payload byte, and XOR check byte. The payload is a signed 16-bit integer in high-byte-first order. The checksum covers the preceding four bytes. Define maximum transaction duration, timeout origin, startup state, repeated-sequence behavior, reset behavior and whether the returned value is new or cached. This is an original teaching protocol, not a standard peripheral format.

A timeout requires a bounded state transition. Record the request as failed, stop accepting its eventual result as current, and use the reset/recovery contract before issuing conflicting work. Do not retry indefinitely inside an input callback. For a controller, a defensible failure policy could mark the input unavailable and release held actions under an explicit game rule; inventing a new joystick sample is not recovery.

Worked timing and decoding example

At a hypothetical 1 MHz serial clock, five bytes require 40 bit periods, or 40 microseconds of wire time. If the device requires 4 microseconds of select setup and 6 microseconds of interphase delay, the best modeled transaction is 50 microseconds before host scheduling overhead. A 40-microsecond timeout would therefore reject a correctly functioning device. Choose a deadline from a bounded worst-case budget, not from payload size alone.

The packet `[165, 0, 0, 5, 160]` decodes to +5 because `165 XOR 0 XOR 0 XOR 5 = 160`. A payload with high and low bytes both 255 decodes to -1: the unsigned value 65535 minus 65536. The check byte detects some corruptions, but XOR is not an authentication mechanism and cannot detect every multi-bit error. A production design chooses its integrity mechanism according to its fault model.

What simulation can and cannot detect

The supplied decoder deliberately receives endpoint mode metadata. It can reject mismatched modes before decoding bytes. Real raw SPI bytes do not generally carry enough information to prove which polarity or phase was used; some wrong-mode transactions may coincidentally appear valid. State this limitation in the report. A logic analyzer or oscilloscope, configured within reviewed limits, can investigate electrical timing that this Python model does not represent.

Start with one bus and one command because it creates a small falsifiable interface. After successful, corrupt, stale and timed-out transactions are distinguishable, add a second command. Preserve an exact trace for each failure. A successful decoder run establishes the behavior of this teaching contract, not compliance of an untested physical peripheral.

Practical assignment and assigned study

Original assignment: Build a simulated peripheral transaction decoder and driver for one interface. Pass: incorrect framing or mode is detected, timeouts are bounded and a complete interface-contract table is provided.

Assigned reading: R53: device drivers, UART, SPI and I2C APIs; R51: bus-level driver documentation. For hardware, add the exact peripheral datasheet.

Run `python -m labs.run demo 36.1` from the package root. Implement `labs/starters/lesson_36_1.py`, then run `python -m labs.run check 36.1 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why can two SPI devices fail at the same clock rate?
2. Does an interface name guarantee voltage compatibility?
3. Why begin with one interface rather than every bus?
4. A five-byte transfer uses a 1 MHz clock plus 10 microseconds of mandatory overhead. Can a 40-microsecond total deadline succeed?
5. Does a valid XOR check prove that the device used the correct SPI mode or that the sender is authentic?

Lesson 36.2 — Firmware, RTOS scheduling and driver lifecycle

Learning outcome: Bound embedded work and make device state transitions explicit.

Real time means meeting a deadline

An RTOS provides mechanisms for scheduling and synchronization; it does not make an arbitrary program timely. Decide which work belongs in an interrupt, which can wait in a worker, and which can be dropped. For a game controller, input sampling may need a short bounded response, while diagnostic logging can be delayed or discarded. Putting both behind one unbounded queue makes their timing inseparable.

Zephyr distinguishes cooperative and preemptive scheduling behavior. A thread that never yields under a cooperative policy can block other work; a preemptive policy still needs bounded interrupt work and a deliberate priority structure. Use the behavior of the pinned RTOS build rather than assuming desktop thread semantics apply. [S02]

Model time before introducing hardware

For each periodic task, record execution demand C, period T, relative deadline D and priority. Utilization `U = sum(C/T)` is a useful load summary, not a general proof of deadlines. Blocking, interrupt arrivals, release jitter and priority assignment can cause a task to fail even when average utilization is below one. Conversely, short simulations do not establish a worst-case response bound outside the modeled assumptions.

The supplied integer-tick simulator uses one core, preemption at tick boundaries, synchronous first releases and lower numbers for higher priority. A task that finishes on its deadline succeeds; unfinished work at that edge counts as a miss. A job may subsequently finish late. These distinctions prevent a common reporting error: counting eventual completions as evidence that all deadlines were met.

Worked schedule and blocking counterexample

Consider input work C=1, T=D=2 at high priority and processing work C=2, T=D=4 at low priority. Across ticks 0 through 3, execute input, processing, input, processing. Three released jobs finish by their deadlines. Total utilization is 1/2 + 2/4 = 1. The model leaves no margin for interrupts, context-switch cost or execution variability, so a real design should not claim this trace proves safe operation.

Now suppose a low-priority logger owns a mutex required by the input task. A medium-priority task can delay the logger while the high-priority input waits. This is priority inversion: runnable priority is not enough when a dependency points at lower-priority work. A protocol such as priority inheritance addresses part of the problem, but logging should usually be removed from the critical input path rather than treated as unavoidable blocking.

The driver lifecycle is another state machine

Use explicit states such as Uninitialized, Ready, Active, Suspended, Removing and Failed. On removal, stop new submissions, arrange cancellation or device quiescence, acknowledge outstanding work exactly once, and reclaim buffers only when the device cannot access them. Completion callbacks can race with

cancellation. A generation number lets the host reject stale completions after reset, but it does not physically stop old DMA. The DMA lifecycle still requires the platform's ownership and synchronization operations. [S05]

A watchdog should represent meaningful forward progress. Feeding it from an unconditional interrupt while the main input pipeline is deadlocked can hide the very failure it is intended to expose. Define a progress condition, such as each critical task completing its required epoch, and service supervision only when that condition is met.

Reason about what remains unmodeled

Use the scheduler to find counterexamples, then document what it omits. In this lesson the model excludes cache misses, mutexes, interrupt nesting and scheduler overhead. The full lab adds driver-removal tests using the ownership model from Lesson 40.3 or an independently written state machine. Keep those tests separate from the periodic-task simulation: passing one does not prove the other.

Practical assignment and assigned study

Original assignment: Use a supported RTOS simulator or a deterministic scheduler model to implement input, processing and logging tasks. Pass: deadlines are evaluated under injected load and removal/cancellation leaves no live invalid requests.

Assigned reading: R53: kernel scheduling, synchronization and device model; R51: driver lifecycle and power management.

Run `python -m labs.run demo 36.2` from the package root. Implement `labs/starters/lesson_36_2.py`, then run `python -m labs.run check 36.2 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Does an RTOS automatically make all tasks meet deadlines?
2. Why not feed a watchdog unconditionally from a stuck loop?
3. What must happen to outstanding requests when a device disappears?
4. A task is released at tick 0 with a deadline at tick 4 and completes on edge 4. Is it late in the supplied model?
5. Why does a reset generation counter not by itself make old DMA buffers safe to free?

Lesson 36.3 — Schematics, PCB layout, power integrity and signal integrity

Learning outcome: Translate a circuit into a manufacturable layout without ignoring physics.

Connectivity is only the beginning

A schematic communicates intended electrical connections. Layout adds physical geometry: component orientation, trace routing, reference planes, vias, clearance and placement. A symbol pin numbered 1 must correspond to the intended package pad numbered 1. A mirrored connector can produce a board that looks tidy, passes selected automated checks, and still connects power to the wrong contact.

KiCad's schematic and PCB tools provide electrical-rule and design-rule checking. Their checks operate on the rules and component data supplied to the project. The lesson's review process therefore treats ERC and DRC reports as evidence to inspect, not a certificate that the circuit is electrically correct. Preserve the actual findings and explain each waiver. [S03; S04]

Create a low-risk first design

Use a low-speed peripheral concept: a connector, a documented low-voltage controller module or selected device, a button/input header, reset provisions and local supply decoupling. Select exact parts and footprints only from their datasheets. The core assignment is a schematic and unrouted or simple two-layer layout in KiCad. No fabrication, purchase or powered board is required. The supplied JSON lint record is a small computational exercise, not a replacement for the schematic.

Separate power domains on paper. Record supply operating range, absolute limits, logic thresholds, maximum pin current and the behavior of unpowered pins. Absolute maximum ratings are not normal operating targets. Add a connector pinout and an orientation view that someone else can check without guessing whether the drawing is viewed from the mating side or component side. Unused or optional connections must be intentional rather than accidental omissions.

Worked power-budget reasoning

Suppose a teaching design budgets 45 mA for a controller, 12 mA for a sensor and 8 mA for indicators. The steady total is 65 mA. A reviewed 100 mA supply budget leaves 35 mA, but startup and pulsed loads may consume more than the average. This is a planning calculation, not a claim about any named part.

A capacitor delivering 40 mA for 2 microseconds with an idealized 40 mV allowed droop requires `C >= I*dt/dV = 2 microfarads`. This charge-balance estimate ignores equivalent series resistance, inductance, regulator dynamics and frequency-dependent behavior. It is useful for exposing an underspecified transient, not for prescribing a universally correct decoupling capacitor. The physical design needs the exact device recommendations and a reviewed return path.

Fast edges make physical loops matter

Signal bandwidth is not determined only by the command rate. A slowly repeated digital pulse can still have a fast edge. Signal current and its return form a loop; interrupting the reference path can enlarge that loop and alter impedance and coupling. A route across a plane split therefore needs more thought than whether two endpoint net names match. Do not treat an arbitrarily long ground lead on an instrument as electrically invisible. Revisit the circuits material for nonideal components and interconnect. [R30]

The first board should remain low speed. High-speed memory and serial links require stackup, impedance, timing and signal-integrity work outside the evidence provided by this simple layout. A successful controller exercise is not a validated graphics-card design. Likewise, the capstone's multiplier-count estimate does not become silicon area merely because it is placed in a hardware report.

Review before manufacturing

Produce a schematic export, annotated connector table, footprint cross-check, board view, power budget and ERC/DRC reports. Ask a qualified reviewer to inspect supply and return paths before any physical extension. A waiver should name the exact finding, explain why it is intentional and identify the responsible reviewer. “Ignore all warnings” is not a review policy. Stop the activity at design review unless the electrical and bring-up plans have been approved for the actual parts and tools.

Practical assignment and assigned study

Original assignment: Create a low-speed peripheral schematic and unrouted or simple two-layer layout in KiCad. Pass: footprints are checked against datasheets, ERC/DRC findings are resolved or justified and a reviewer inspects power and return paths before fabrication.

Assigned reading: R36: schematic editor, PCB editor, ERC and DRC; R30: nonideal circuit behavior; R49: interconnect and packaging extension.

Run `python -m labs.run demo 36.3` from the package root. Implement `labs/starters/lesson_36_3.py`, then run `python -m labs.run check 36.3 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Does passing DRC prove electrical correctness?
2. Why do return paths matter?
3. Why verify footprints against datasheets?
4. A 65 mA steady load fits a 100 mA budget. What information is still needed before approving the supply?
5. Is the supplied board-review JSON checker sufficient to complete the original KiCad assignment?

Lesson 36.4 — Bring-up, instrumentation and fault isolation

Learning outcome: Turn an unresponsive device into a sequence of testable hypotheses.

Replace random changes with discriminating tests

An unresponsive peripheral is not a diagnosis. “Firmware bug” is one hypothesis among power, reset, clock, connection, protocol and identity failures. Start with prerequisites and make each observation eliminate specific possibilities. Changing firmware, wiring and clock configuration simultaneously may make the device appear to work while destroying the evidence needed to explain why.

The core exercise uses a simulated device. Physical extensions must use the exact board documentation, approved supply, instrument limits and a reviewed procedure. Do not probe unknown high-voltage circuits, defeat instrument grounding, or bypass protection to force a response. Record explicit stop conditions before powered work. The curriculum does not authorize unsupervised electrical bring-up.

Build an expected-observation table

Each row should contain a hypothesis, the least invasive observation, the expected result, the branch when that result is absent, and the safe stop condition. For example, “reset remains asserted” predicts no useful application response even when supply and clock are valid. Checking the reset state is more discriminating than repeatedly trying a complicated game-input command.

Use the ordering safety limits, power, reset, clock, basic identity, simple transaction, then application behavior. The exact physical procedure can change by device; the ordering is an engineering dependency model, not universal instructions to apply power or touch a probe. The teaching decision tree returns actions such as `REVIEW_CLOCK` rather than performing them.

Worked fault-isolation session

In the simulated case, supply and reset are acceptable, the clock is absent, and device identity reads incorrectly. The first useful branch is the missing clock. The failed identity read is expected downstream of that fault and does not yet imply a corrupted device register. Restore the modeled clock, rerun the identity command, and then introduce a separate framing fault. Keeping these experiments distinct tests whether the diagnostic procedure follows dependencies.

Now inject an over-limit observation while the protocol also fails. The correct branch is `STOP_AND_ISOLATE`, not “try a slower clock.” A diagnostic procedure must never use a downstream functional failure as a reason to ignore an upstream safety violation. The returned status records that the session stopped; it does not claim the underlying fault has been repaired.

Instruments answer different questions

A logic analyzer is useful for recorded digital transitions and protocol decoding; an oscilloscope can reveal analog waveform behavior within its capabilities. A debugger observes selected execution state and may change timing. A serial log can be delayed, truncated or produced by the wrong firmware build. None is a universal truth source. Before using an instrument physically, study its manual, input limits and grounding requirements; choose the measurement that can actually distinguish the competing hypotheses.

Keep firmware and configuration identity in every trace. Record the binary hash or source revision, clock settings, device reset epoch and any tool-side decoding assumptions. A perfect-looking trace decoded with an incorrect mode is not strong evidence. For a purely simulated run, record the injected fault and model version rather than inventing a hardware serial number.

Rehearse the unhappy paths

A good bring-up plan includes what to do after a missing expected observation: stop, inspect a disconnected design, recover a model, or request review. It also includes conditions for restarting a session and for retaining a failed unit rather than repeatedly cycling it. Treat “unknown observation” as its own result. A decision tree that classifies every unfamiliar trace as firmware failure is not fault isolation.

Finish with a short incident note: symptom, tested hypotheses, observed evidence, root cause if established, change made, and regression test. Separating “confirmed,” “consistent with,” and “not yet tested” prevents a plausible explanation from becoming an unsupported fact. This habit carries directly into engine crash analysis and performance investigations later in the volume.

Practical assignment and assigned study

Original assignment: Write and rehearse a bring-up procedure against a simulated device with injected clock, reset and protocol faults. Pass: each fault has a safe diagnostic branch and the procedure includes explicit stop conditions.

Assigned reading: R53: board support and debugging; R36: fabrication outputs and design review; selected instrument manuals for any physical extension.

Run `python -m labs.run demo 36.4` from the package root. Implement `labs/starters/lesson_36_4.py`, then run `python -m labs.run check 36.4 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why test power, clock and reset before complex application behavior?
2. What is the value of an expected-observation table?
3. What should happen when a physical observation exceeds a reviewed safe limit?
4. The identity register fails while the modeled clock is missing. Which branch should the procedure inspect first, and why?
5. Why must a trace identify the firmware build and decoder configuration?

Module 37 — GPU microarchitecture and memory systems

Prerequisites: Modules 16, 22, 32 and 34.

Original module project: Create a GPU architecture workbook connecting shader behavior to execution groups, registers, memory transactions and fixed-function resources. Measure one accessible GPU or clearly label a simulator-only study.

Lesson	Learning outcome
37.1 · SIMT execution, divergence, occupancy and latency hiding	Explain why many threads do not guarantee useful parallel work.
37.2 · Coalescing, local storage, caches and shared-memory banks	Match data layout to the memory transactions actually performed.
37.3 · Raster, texture, matrix and ray-tracing units	Understand fixed-function acceleration as specialized tradeoffs.
37.4 · CPU-GPU sharing, discrete memory and synchronization	Treat shared addressability separately from ownership and cost.

Read the module-project brief in the workbook before beginning. The four lessons build evidence for that project rather than four unrelated demonstrations.

Lesson 37.1 — SIMT execution, divergence, occupancy and latency hiding

Learning outcome: Explain why many threads do not guarantee useful parallel work.

Threads are a programming abstraction; issued work consumes resources

A shader invocation looks like an independent computation, but hardware schedules work in groups. SIMT exposes thread-level control flow while sharing execution machinery. NVIDIA's CUDA documentation describes warps and thread blocks; other platforms and APIs use their own subgroup and workgroup contracts. Query supported behavior and record the chosen device rather than assuming every GPU has one universal group width. [S06]

A group can contain lanes that follow different branches. In a simplified serialized-path model, the machine issues each necessary path while inactive lanes do no useful work on that path. This can lower active-lane efficiency even though every invocation eventually produces the correct result. A branch that is coherent across a group behaves differently from a branch whose predicates alternate lane by lane.

Derive a useful-work model

Suppose a four-lane educational group has two lanes on path A and two on path B. Each path requires four issue slots, with no common prefix. The group issues eight slots, exposing 4*8=32 lane-slot opportunities, but only 16 do useful work. Efficiency is 50%. If all four lanes choose A, only four slots are issued and all 16 lane-slot opportunities are useful. This metric is deliberately not wall-clock throughput.

Add four common slots before the branch. The split group now uses 32 useful lane-slots out of 48 opportunities, or 66.7%. The branch remains divergent, but its effect on total useful work is smaller. If only one lane is active because the input ends in a partial group, that tail is another source of underutilization even with coherent control flow.

Occupancy is a capacity ratio, not a success score

Occupancy commonly describes resident active work relative to a hardware limit. Registers, shared local storage, workgroup size and scheduling limits constrain residency. More resident groups can hide some latency when one group waits and another has independent work. But residency alone cannot remove a dependency chain, generate missing memory bandwidth or make an expensive instruction cheap. Reducing register allocation may introduce spilling or recomputation. [S07]

For a hypothetical processor with 65,536 registers, 2,048 resident-thread capacity and 256-thread blocks, 32 registers per thread permit eight blocks by the register limit. At 64 registers per thread, the register limit permits four blocks. That changes the simplified thread occupancy from 100% to 50%. Real allocation granularity and other limits may reduce these counts; they are arithmetic examples, not properties of a named GPU.

Now consider the 50%-occupancy kernel completing faster because its additional registers preserve reused values instead of loading them repeatedly. That is not a contradiction. Occupancy describes one resource relationship; measured time reflects the complete execution. Conversely, a high-occupancy kernel with poor coalescing can keep many groups waiting for the same memory bottleneck.

Turn a hypothesis into a controlled comparison

Construct coherent and divergent variants that perform the same intended useful operation and produce matching outputs. Record input distribution, group width, group size, compiler settings and numerical tolerance. Change one factor at a time. Sorting work to improve coherence may help execution, but sorting and restoring order belong in the end-to-end cost.

The supplied model counts issue slots and lane use only. Its output says nothing about a real scheduler, reconvergence implementation or exact instruction timing. The simulator-only route therefore reports structural efficiency and hypothetical capacity. The optional GPU route records actual device properties and timings. Neither route should equate a better efficiency percentage with a proven faster frame until the application experiment establishes that connection.

Practical assignment and assigned study

Original assignment: Compare coherent and divergent workloads, varying register pressure or group size where supported. Pass: outputs agree, device group assumptions are recorded and the report does not equate occupancy with performance.

Assigned reading: R37: programming model, execution and hardware implementation; R48: vector and multithreaded architecture foundations.

Run `python -m labs.run demo 37.1` from the package root. Implement `labs/starters/lesson_37_1.py`, then run `python -m labs.run check 37.1 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. What is branch divergence in grouped GPU execution?
2. Does high occupancy guarantee high throughput?
3. Why can reducing register use sometimes hurt?
4. In a four-lane group with two lanes on each of two four-slot paths and no common prefix, what is useful lane-slot efficiency?
5. A new kernel lowers occupancy from 100% to 50% but runs faster. Is the measurement inherently invalid?

Lesson 37.2 — Coalescing, local storage, caches and shared-memory banks

Learning outcome: Match data layout to the memory transactions actually performed.

Count traffic at a declared level

Source code asks for elements; hardware services transactions. A memory instruction accessing neighboring bytes can use fewer transaction units than the same useful work scattered across distant addresses. Coalescing is therefore a relationship among the addresses requested together, their sizes, alignment and the memory-system rules. It is not simply a synonym for “use arrays.” NVIDIA's documentation provides concrete global-memory and shared-memory examples for its supported architecture model. [S06]

Declare the level: logical bytes requested, unique bytes needed, cache-sector traffic, external-memory traffic or host-device payload. They are not interchangeable. A cache hit can prevent an external transaction, and repeated lanes reading one location can have a large logical request count but a small set of unique useful bytes. The supplied model deliberately counts sectors for one instruction, not actual DRAM traffic over an application.

Worked contiguous, strided and misaligned accesses

Assume 32 lanes, four-byte words and 32-byte sectors. Lanes requesting addresses 0,4,...,124 touch four sectors and use all 128 transferred bytes in this simplified model. Lanes requesting 0,32,...,992 touch 32 sectors: 1024 transferred bytes for 128 useful bytes, giving 12.5% utilization. These results follow from the declared address mapping, not from an assumed measured bandwidth.

Misalignment also matters. One four-byte word starting at address 30 touches bytes 30 through 33 and crosses two sectors. Counting only the start address would report one sector and undercount traffic. That boundary case is included in the starter tests. Repeated addresses are handled as unique-byte sets so that a broadcast does not falsely appear to transfer the same payload many times.

Tiling trades movement for storage and coordination

A tiled algorithm stages a bounded working set close to the executing threads and reuses it before eviction. The benefit must exceed staging instructions, synchronization and pressure on local resources. A tile that increases reuse but consumes too much shared storage can reduce resident work. A transpose can need different layouts for global coalescing and shared-bank access; good layout at one level does not automatically remain good at another.

For an educational shared-memory model with 32 banks and one 32-bit word per bank index, word address w maps to `w mod 32`. Reading a column of a 32-word-wide tile gives addresses 0,32,64,... and maps different words to the same bank. Padding the row width to 33 maps consecutive column entries to successive banks. Repeated reads of the same word are a broadcast case in the selected model, not 32 different words competing for one bank. Writes require their own correctness rules. [S06]

Names can mislead

In CUDA, “local memory” is thread-local in scope, not a promise that every value lives in fast on-chip storage. Other APIs use similar words differently. State address space, physical resource assumptions and lifetime explicitly. A register spill can alter memory traffic without appearing as a new array in the high-level algorithm. [S06; S07]

Suppose a structure-of-arrays conversion costs 0.7 ms and reduces a kernel from 1.0 ms to 0.6 ms. For one invocation, the converted route costs 1.3 ms and loses. If the converted representation can be reused for ten invocations without invalidation, its total is 0.7 + 10*0.6 = 6.7 ms versus 10 ms. Representation lifetime is part of the optimization, not bookkeeping to exclude.

Evidence that explains more than a bar chart

Predict sectors and bank conflicts before running. Validate identical useful outputs, include partial tiles and irregular sizes, and distinguish analytical traffic counts from measured counters. A result below a theoretical bandwidth ceiling is not enough to name a memory bottleneck; dependencies and instruction overhead may dominate. Use the traffic model to formulate the next controlled experiment rather than treating it as a complete GPU simulator.

Practical assignment and assigned study

Original assignment: Benchmark contiguous, strided and tiled access with identical useful work. Pass: count useful bytes, state the relevant memory level and include any layout-conversion cost in the application comparison.

Assigned reading: R37: device memory access, shared memory and performance considerations; R43: operational intensity.

Run `python -m labs.run demo 37.2` from the package root. Implement `labs/starters/lesson_37_2.py`, then run `python -m labs.run check 37.2 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. What does coalescing improve?
2. Why can shared memory still cause stalls?
3. Why include layout conversion in a comparison?
4. Why does a four-byte access at byte address 30 touch two 32-byte sectors?
5. When can a 0.7 ms conversion followed by a 0.6 ms kernel beat an original 1.0 ms kernel?

Lesson 37.3 — Raster, texture, matrix and ray-tracing units

Learning outcome: Understand fixed-function acceleration as specialized tradeoffs.

Specialized units accelerate selected operations

A GPU combines programmable execution with resources specialized for particular forms of work. Rasterization, texture sampling, matrix operations and ray-related operations are not interchangeable throughput currencies. A unit is useful when the workload reaches it in the required format, with supported precision and appropriate work size. Feature availability and numerical behavior come from the selected API and device contract, not from a marketing label. [R21; R23; S07]

A ray-related unit does not eliminate scene construction, acceleration-structure updates, material evaluation, lighting integration, denoising or output work. A matrix unit does not remove the need to arrange data and accept its numerical format. Texture hardware does not make a texture access independent of cache behavior, filtering choices or memory demand. The co-design question is which part of the full path becomes cheaper and what costs appear elsewhere.

Make resource demands explicit

Write a workload vector: arithmetic operations, texture samples, ray operations, bytes moved and setup work. Pair each demand with an applicable service-rate assumption. In an ideal-overlap model, the maximum of the resource service times is a lower bound on elapsed service, while the sum is a useful no-overlap comparator. Neither is a complete timing prediction when dependencies, contention and queues are unspecified.

For an original teaching case, arithmetic demand requires 1.0 ms at its assumed rate and texture service requires 0.2 ms. Setup adds 0.1 ms. Ideal overlap gives 1.1 ms; entirely serial service gives 1.3 ms. If arithmetic capacity doubles, the ideal model becomes 0.6 ms. If instead texture service takes 2.0 ms, accelerating arithmetic from 0.1 to 0.05 ms barely changes the bottleneck. These are hypothetical service rates, not claims about a commercial device.

Precision and quality are part of useful work

Comparing a lower-precision matrix path with a full-precision scalar path is meaningful only after declaring an acceptable error criterion. Report absolute error, relative error where the reference is not near zero, and application-specific consequences. A small arithmetic error can become visible after repeated integration, thresholding or reconstruction. Conversely, exact arithmetic may be unnecessary for a purely visual quantity with a justified tolerance.

Keep numerical correctness separate from visual quality. A render can be numerically different yet acceptable under the intended quality target; it can also have a low mean error while containing a rare unacceptable artifact. Design adversarial inputs and inspect the distribution of error rather than selecting one favorable screenshot. For the model-only route, use an explicit quality proxy and describe its limitations.

Setup can reverse a small-workload result

Suppose general-purpose execution costs 0.2 microseconds per item. A specialized route costs 0.05 microseconds per item plus 20 microseconds of setup. The specialized path wins only when `20 + 0.05N < 0.2N`, giving `N > 133.33`. The first integer size that wins in this simplified model is 134. Transfers or format conversion can move the crossover further right.

This derivation is more useful than saying the unit is “4x faster”: the arithmetic slope is four times better, but many small requests lose. Batching may amortize setup while increasing queueing delay for interactive work. A design must choose latency and throughput requirements explicitly rather than improving one while silently violating the other.

Investigate one workload deeply

Select a texture-heavy, arithmetic-heavy or ray-query case. Start with a documented resource model and a competent implementation. Record the quality target, feature assumptions, batch sizes and included phases. Change one resource demand and observe whether the expected bottleneck moves. The supplied resource calculator helps test arithmetic consistency; it cannot identify a real device bottleneck without measured evidence or a justified architectural model.

Practical assignment and assigned study

Original assignment: Analyze one texture-heavy, arithmetic-heavy or ray-query workload. Pass: identify the likely resource limits using measurements or a documented model and state supported precision and feature assumptions.

Assigned reading: R37: architecture and specialized operations; R21: graphics/compute capabilities; R23: ray intersection and acceleration reference.

Run `python -m labs.run demo 37.3` from the package root. Implement `labs/starters/lesson_37_3.py`, then run `python -m labs.run check 37.3 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Does a ray-tracing unit eliminate the rest of rendering work?
2. Why compare the same numerical precision or quality?
3. When might specialized hardware lose on a small workload?
4. A path costs 20 microseconds setup plus 0.05 microseconds per item; the baseline costs 0.2 microseconds per item. At what first integer size does it win?
5. Why is the sum of independent resource-service estimates not a guaranteed upper bound on observed latency?

Lesson 37.4 — CPU-GPU sharing, discrete memory and synchronization

Learning outcome: Treat shared addressability separately from ownership and cost.

Addressability, coherence, ordering and ownership are different

Two processors seeing the same address does not imply they may safely modify it at the same time. Addressability concerns how memory is named. Coherence concerns the relationship between cached copies. Ordering concerns which operations can become visible before others. Ownership specifies which agent may read, write, reuse or free a resource during each phase. A correct design needs all four questions answered.

Linux's DMA documentation distinguishes CPU virtual, physical and DMA/bus addresses, and explains that coherent mappings do not remove every ordering requirement. Vulkan likewise exposes explicit synchronization responsibilities. The operations required by a real implementation depend on the selected API and platform; a generic C++ pointer or `volatile` annotation is not a substitute for those contracts. [S05; S09]

Define the complete request path

Use states HostWritable, Submitted, DeviceOwned, Completed and HostReadable. The host prepares input, publishes the descriptor under the correct visibility rules, submits work, observes the required completion signal, and only then consumes or reuses output. A fence indicating completion and a resource transition expressing visibility or usage are related but not universally interchangeable. Record the exact API meaning rather than assuming every “wait” flushes every cache.

For a discrete-memory route, count allocation or reuse policy, input preparation, upload, dispatch, kernel execution, readback, synchronization and result consumption. Persistent device data may eliminate some transfers, but only when its lifetime and consumers actually allow that. For a shared-memory route, explicit copies may disappear while coherence traffic, cache maintenance, page behavior, synchronization or contention still cost time. [S08; S21]

Worked transfer comparison

Assume 1 MiB of input and 1 MiB of output, with an effective host payload bandwidth of 8 GB/s. The two transfers require `2*1,048,576 / 8e9 = 0.262144 ms`. Add 0.05 ms setup, 0.20 ms compute and 0.03 ms completion: one serialized request costs 0.542144 ms. Use binary MiB for payload and decimal GB/s for bandwidth; silently treating both as the same prefix introduces a numerical error.

A hypothetical shared route with the same compute, setup and completion but 0.04 ms of visibility overhead costs 0.32 ms. That is a result under the supplied assumptions, not proof that every shared architecture is faster. If CPU and GPU contend for the same memory system, compute time may change too. The supplied calculator keeps compute service separate and does not double-count internal kernel traffic as an additional copy.

Pipelining changes throughput differently from latency

With independent upload, compute and download resources, multiple requests can overlap. Their steady-state interval is constrained by the slowest stage and any shared resource bottleneck; a single request still traverses its required stages. Double buffering enables overlap only when the two buffers have unambiguous ownership. Reusing a buffer one frame early can produce intermittent corruption rather than a deterministic crash.

A timeout does not necessarily return ownership. If the device may still access a buffer, quarantine it until completion under the declared policy or an acknowledged quiescence/reset. Merely abandoning the wait can leave the hardware writing into a buffer that the application has reassigned. Lesson 40.3 makes this failure path executable.

Measure the same boundary for both designs

Record phase timings and a separate end-to-end measurement. GPU timestamp intervals and host clock intervals may use different clocks and scopes; do not subtract them without a supported calibration relationship. State whether the final consumer actually needs a readback. An accelerator whose output stays on the GPU solves a different transfer problem from one whose output must immediately affect CPU-authoritative gameplay.

Practical assignment and assigned study

Original assignment: Compare an upload/compute/readback path with an available shared-memory path, or model both with stated assumptions. Pass: correct ownership, separate phase timings and no claim that unified means zero-cost.

Assigned reading: R37: memory management and synchronization; R21, R24 or R25: the selected platform's resource-sharing contract.

Run `python -m labs.run demo 37.4` from the package root. Implement `labs/starters/lesson_37_4.py`, then run `python -m labs.run check 37.4 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Does shared addressing mean zero data-access cost?
2. Why is CPU modification during GPU use risky?
3. What must an end-to-end accelerator benchmark include?
4. Why can a timed-out request still own its buffer?
5. Does double buffering automatically halve single-request latency?

Module 38 — Quantitative performance engineering

Prerequisites: Modules 09, 16, 18, 30 and 37.

Original module project: Produce a performance investigation with a hypothesis, controlled workload, raw measurements, uncertainty, a bottleneck model and a verified optimization. A negative result is acceptable when the evidence is rigorous.

Lesson	Learning outcome
38.1 · Amdahl, roofline and bottleneck models	Use upper bounds to reject implausible optimization claims.
38.2 · Profilers, counters and trustworthy benchmark design	Distinguish observation from an explanation that still needs testing.
38.3 · SIMD, cache blocking, NUMA and job scheduling	Optimize locality and parallelism together.
38.4 · Energy, thermal behavior and sustained performance	Evaluate useful work over time rather than only a short peak.

Read the module-project brief in the workbook before beginning. The four lessons build evidence for that project rather than four unrelated demonstrations.

Lesson 38.1 — Amdahl, roofline and bottleneck models

Learning outcome: Use upper bounds to reject implausible optimization claims.

A bound is useful before an optimization exists

Suppose a frame takes 20 ms and a candidate subsystem takes 4 ms on its critical path. Even removing that subsystem completely leaves 16 ms: the best possible speedup under this additive model is 20/16 = 1.25. A 10x subsystem improvement is not a 10x game improvement. First identify the boundary and the fraction of that boundary affected. Overlapping CPU and GPU work requires a dependency timeline rather than adding every measured duration indiscriminately.

Write Amdahl's model as `speedup = 1 / (s + (1-s)/p + h)`. Here `s` is the unchanged fraction of original time, `p` is the speed multiplier for the changed portion, and `h` is new overhead expressed as a fraction of original total time. The model assumes fixed useful work. It does not describe a developer spending the savings on more particles or a larger scene. With s=0.2 and p=4, time becomes 0.4 of the original and speedup is 2.5. Adding h=0.1 reduces speedup to 2.0. As p grows without limit and h=0, the unchanged 20% imposes a 5x ceiling.

Worked example: the original roofline bound

The original lesson's machine has a declared compute ceiling of **1,000 GFLOP/s**, a declared bandwidth of **100 GB/s**, and a kernel intensity of **2 FLOP/byte**. Using decimal gigabytes, the bandwidth ceiling is 100 × 2 = 200 GFLOP/s. Therefore `P <= min(1000, 200) = 200 GFLOP/s`. The ridge point is 1000/100 = 10 FLOP/byte. This is a hypothetical numerical example, not a measurement of a specific GPU. The Roofline model connects arithmetic intensity with compute and memory ceilings. [S10; R43]

The memory level is part of the equation. An intensity counted at DRAM cannot be paired with an L1-cache bandwidth. Count the actual transfers relevant to the declared level, including necessary output traffic and, where applicable, write allocation or redundant loads. Count arithmetic consistently: state whether a fused multiply-add contributes two floating-point operations. Compression, caching, vector width and precision can change the applicable ceilings; a marketing peak is not automatically a measured sustainable limit.

Diagnose below the roof, not only above it

A kernel below the bound might be limited by dependency chains, insufficient parallel work, synchronization, irregular accesses, dispatch overhead or a different resource. The roof is not a promise. Being below it does not prove the model wrong, and moving closer to it does not prove that frame latency improved. Compare a hypothesis with a targeted experiment: change reuse while holding numerical work constant, or increase batch size while holding layout constant. Then inspect whether the predicted trend occurs.

A point apparently above the roof deserves a units-and-boundary audit before celebration. Perhaps bytes were counted at the wrong level, a peak was underestimated, elapsed time omitted asynchronous completion, or useful arithmetic was counted twice. Keep the uncorrected observation and record the correction so another reader can follow the investigation.

Use the model to choose the next experiment

The supplied calculator verifies algebra and units. It cannot discover a machine's bandwidth or a game's critical path. Your full worksheet must record the source of each parameter, the uncertainty or plausible range, and the result when an optimistic assumption weakens. If the entire plausible range predicts negligible application benefit, a documented decision not to build an accelerator is valuable engineering work.

Practical assignment and assigned study

Original assignment: Build a roofline worksheet for one kernel and an Amdahl estimate for one frame optimization. Pass: units cancel correctly, modeled ceilings use the declared memory level and observed deviations are explained as hypotheses.

Assigned reading: R43: original Roofline model; R48: quantitative architecture; revisit Module 18 parallel limits.

Run `python -m labs.run demo 38.1` from the package root. Implement `labs/starters/lesson_38_1.py`, then run `python -m labs.run check 38.1 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. What is the example roofline bound?
2. What is the maximum speedup with 20% unchanged serial work and infinitely fast remaining work?
3. Does being below the roof prove the model is wrong?
4. A kernel is 10% of a frame and becomes twice as fast without overhead. What is frame speedup?
5. Why must intensity and bandwidth refer to the same memory level?

Lesson 38.2 — Profilers, counters and trustworthy benchmark design

Learning outcome: Distinguish observation from an explanation that still needs testing.

Define the observation before collecting it

“Faster” must have a numerator, denominator and boundary. State useful workload, input distribution, implementation version, compiler options, timing clock, warmup policy, repetitions, validation method and exclusion rule before running the comparison. A host timer around an asynchronous dispatch may measure submission rather than completion. Allocation outside a kernel-only interval is legitimate when declared, but that interval is not an end-to-end application measurement.

Use a monotonic clock for elapsed time, and make intervals large enough relative to timer resolution and invocation overhead. Python's timeit documentation discusses repeated timing and setup boundaries; its convenience does not make a Python loop a proxy for native GPU execution. The provided Python exercises teach analysis and correctness. The optional C++ capstone baseline separately records native host measurements. [S11]

A benchmark must reject a plausible wrong answer

A no-op can be extremely fast. So can a vectorized path that drops the final three particles or an approximate path that exceeds the quality limit. Compare outputs with a trusted oracle before accepting timings, and keep validation outside the timed kernel interval only when the boundary explicitly excludes it. A checksum guards against some mistakes and dead-work elimination; a full comparison on representative and adversarial inputs is stronger. A checksum collision or consistently incorrect oracle can still escape.

The native teaching baseline uses two contiguous-memory software variants, alternating their order across repeated samples. It checks results against an independent exact-double oracle and records raw CSV data. It does not prove optimality on every CPU. A blocked loop is a candidate, not a guaranteed improvement, and a virtualized host is not a controlled target gaming platform.

Worked example: tails and policies change interpretation

For retained durations `[10, 10, 11, 12, 40]` ms, mean is 16.6 ms and median is 11 ms. Under this course's nearest-rank definition, p95 is the element at rank ceil(0.95 × 5)=5, or 40 ms. The maximum is therefore also the estimated p95 in this tiny dataset. This does not establish the population's 95th percentile with high confidence. Report the sample count and collect enough independent runs for the intended conclusion.

A predeclared warmup exclusion addresses a named experimental condition. Removing the 40 ms sample only because it is inconvenient changes the question after seeing the result. Preserve all raw records and attach an exclusion reason. If another process demonstrably interrupted a trial and your protocol allows that exclusion, show both the inclusive result and the declared filtered result.

Counters support hypotheses, not magical diagnoses

Counter names, collection scopes and interpretation depend on the processor and tool. State exactly which event was measured, whether collection perturbed execution, and whether multiple counters were multiplexed. A high cache-miss count is not by itself proof that cache misses control frame time: overlap, useful work and critical-path location matter. Use a controlled layout or locality change to test causality. Official vendor guidance is the authority for the selected counter's meaning. [S07; R37]

For a comparison, keep workload and quality equal, interleave or randomize order, repeat whole experiments where practical, and report uncertainty rather than only the best sample. Paired trials can reduce shared drift when their pairing is meaningful. Confidence intervals require an explicit sampling model; thousands of adjacent correlated frame times are not automatically thousands of independent observations. Your report should explain what the evidence cannot distinguish, not just which bar is shorter.

Practical assignment and assigned study

Original assignment: Create a benchmark harness with warmup, repeated runs, output validation and metadata. Pass: an intentional no-op or wrong-result variant is rejected and all included/excluded samples are documented.

Assigned reading: R26: profiling; R37: performance analysis; R09: inference and uncertainty.

Run `python -m labs.run demo 38.2` from the package root. Implement `labs/starters/lesson_38_2.py`, then run `python -m labs.run check 38.2 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why validate benchmark output?
2. Are similarly named counters identical across all processors?
3. Why declare an exclusion policy before inspecting results?
4. For five samples 10,10,11,12,40 ms, what are median and nearest-rank p95?
5. Can a kernel-only measurement legitimately exclude allocation?

Lesson 38.3 — SIMD, cache blocking, NUMA and job scheduling

Learning outcome: Optimize locality and parallelism together.

Locality and parallelism share a budget

A particle update can be limited by arithmetic, memory movement, job overhead or imbalance. Adding threads helps only the part that can execute concurrently without creating a larger cost elsewhere. Begin with a competent sequential loop, predictable storage, stable inputs and verified outputs. Change one structural variable at a time so the cause of an improvement remains interpretable.

An array of structures keeps each object's fields together; a structure of arrays keeps one field across many objects together. A velocity-only pass may benefit from loading contiguous velocity values without unrelated fields. A pass consuming every field may have different tradeoffs. Count conversion and synchronization when representations differ across subsystems. CPU vectorization also depends on aliasing information, alignment, dependencies and arithmetic semantics. Tail elements must be processed when the count is not a multiple of the vector width. Reassociation of floating-point operations can change rounding, so test against the declared quality budget rather than assuming bit identity. [R11; R48]

Blocking reduces the working set, not the problem size

A tile improves reuse when the data needed together fits the relevant storage and is revisited before eviction. For a simple dense multiply tile, count all participating arrays, accumulator storage and live temporaries, not just one input. A nominal cache capacity is not fully available to a single tile: associativity, other threads and unrelated engine work compete. The useful tile size is an experimental choice constrained by a model.

False sharing occurs when threads update different logical objects that occupy the same coherence unit, causing ownership movement despite no intentional sharing of those objects. Padding every object indiscriminately can increase traffic and memory footprint. Prefer per-worker accumulation followed by a controlled reduction when the operation permits it; then inspect layout and actual cache-line assumptions. NUMA adds placement: a page's physical location and the worker accessing it can matter. First-touch behavior and affinity are operating-system and policy dependent, not universal promises. Document the machine before asserting a locality diagnosis. [R48]

Worked example: equal job counts are not equal work

Take ready jobs with costs `[8,1,8,1]` time units on two identical workers and no dispatch overhead. Round-robin assignment gives worker A costs 8+8=16 and worker B costs 1+1=2: makespan is 16. Greedy placement on the earliest available worker gives final loads 9 and 9: makespan is 9. This is a deterministic scheduling model, not evidence that a particular work-stealing implementation achieves 9 units on hardware.

Add a per-job cost of 0.5 and the model changes. Making jobs smaller can improve balance while increasing dispatch, queue and synchronization costs. When a job has dependencies, “earliest available” is constrained by readiness, not just worker load. A parallel update with a final reduction also waits for its slowest required predecessor. Overlap and contention should be drawn explicitly.

Optimize both ends of the workload distribution

Test zero items, a single item, a vector-width tail, a cache-sized batch and a large batch. The dispatch cost that disappears in a million-particle test can dominate a small menu animation. Record numerical differences, allocation behavior and memory footprint along with timing. A fallback threshold can be justified when small workloads prefer the sequential path; the threshold itself needs sensitivity testing.

The supplied slice compares job assignments. It does not implement SIMD, operating-system threads or NUMA placement. The full assignment adds a real locality change and scheduling change to a particle or transform workload. Use compiler vectorization reports where available and label unsupported mechanisms as hypotheses, not as measurements obtained from this model.

Practical assignment and assigned study

Original assignment: Optimize a particle or transform workload using one locality change and one scheduling change. Pass: test tail sizes, report numerical differences, and measure small as well as large workloads to expose overhead.

Assigned reading: R37: parallel memory patterns; R48: multiprocessors and memory systems; R11: safe C++ and concurrency.

Run `python -m labs.run demo 38.3` from the package root. Implement `labs/starters/lesson_38_3.py`, then run `python -m labs.run check 38.3 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. What is false sharing?
2. Why can very small jobs be inefficient?
3. Why test small workloads after a parallel optimization?
4. For ready costs 8,1,8,1 on two workers, compare round robin and greedy makespan with no overhead.
5. Does changing to structure-of-arrays always improve performance?

Lesson 38.4 — Energy, thermal behavior and sustained performance

Learning outcome: Evaluate useful work over time rather than only a short peak.

Power is a rate; energy is accumulated work cost

Watts are joules per second. A 50 W workload lasting 2 seconds consumes **100 J** within the stated measurement boundary. An 80 W workload lasting 1 second consumes **80 J**. In this original example, the higher-power task uses less task energy. The comparison ignores transitions and post-task idle energy; include those when evaluating battery life over a full usage cycle.

For interval-average power samples, integrate `E = sum(P_i × delta_t_i)`. If the samples instead represent instantaneous power at endpoints, a trapezoidal estimate is a different assumption. Do not silently treat one data format as the other. Record timestamps, units, sample meaning, missing intervals and sensor scope. Whole-device wall power, processor-package telemetry and GPU telemetry are not interchangeable measurements. A baseline subtraction can estimate incremental energy, but it inherits the baseline's uncertainty and may not isolate a component cleanly.

Worked example: useful work changes the conclusion

Suppose an early interval renders 600 acceptable frames in 10 s at 60 W average. Energy is 600 J and efficiency is 1 frame/J. A later interval renders 450 acceptable frames in 10 s at 50 W: energy is 500 J and efficiency is 0.9 frame/J. Later power is lower, but useful-frame rate and frames per joule are worse. These are synthetic teaching values, not a trace from a real handheld.

Define “acceptable” before using such a metric. A renderer that halves quality below the agreed threshold has not necessarily improved efficiency. Report both useful-frame rate and quality, with separate simulation correctness. An authoritative RTS simulation should not change its rules because a thermal policy reduces rendering resolution.

Sustained performance is a different experiment

A short run may end before temperature, frequency, fan response or resource management reaches a representative state. Record the workload duration and compare early and sustained windows under the normal operating policy. Ambient conditions, charging state, background applications and device orientation can alter a physical experiment. Repeat the conditions that matter to the intended use. Android's public performance framework provides thermal and workload-related mechanisms; actual behavior and available APIs must be checked for the selected device and OS. [S12]

A sensible adaptive renderer can reduce optional work with hysteresis: enter a lower tier after a sustained constraint, and return only after a stronger recovery condition. Hysteresis avoids rapid oscillation between modes. The policy still needs tests for resource allocation spikes, state continuity and the time required to switch. A thermal signal is not a universal calibrated measurement of junction temperature.

Keep normal protections in place

Do not disable temperature, current or power protections for this course. Physical measurement is optional, and the supplied synthetic trace supports the core reasoning. Where a real device is used, stay within its documented operating limits and stop a test that behaves abnormally. A controlled negative result is more useful than an unsafe “peak” number.

The lab calculator integrates interval-average power and returns energy and useful frames per joule. Its arithmetic cannot establish sensor accuracy, validate a thermal model or predict a product's battery life. The full report must distinguish measured values from generated data and state which extra measurements would be required for a purchasing or engineering decision.

Practical assignment and assigned study

Original assignment: Run a sustained workload within normal device limits or analyze a synthetic power trace. Pass: distinguish watts from joules, report the measurement boundary and compare early versus sustained performance.

Assigned reading: R48: quantitative architecture; R51: device power management; R37: GPU execution and performance context.

Run `python -m labs.run demo 38.4` from the package root. Implement `labs/starters/lesson_38_4.py`, then run `python -m labs.run check 38.4 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. How much energy is 50 W for 2 seconds?
2. Which example uses less task energy?
3. Why not disable thermal protections for a curriculum benchmark?
4. Why distinguish interval-average from instantaneous power samples?
5. Can lower watts mean worse efficiency?

Module 39 — Gaming-platform constraints and interactive latency

Prerequisites: Modules 22, 27, 29 and 36-38.

Original module project: Create a platform capability and latency report for the engine. Implement one non-desktop constraint in simulation or on an available device: touch input, suspend/resume, restricted memory, variable presentation or XR timing.

Lesson	Learning outcome
39.1 · Input-to-display latency and frame pacing	Account for the whole interaction path rather than only frame rate.
39.2 · Mobile and handheld systems: memory, power and lifecycle	Design for suspension, limited budgets and sustained use.
39.3 · XR spaces, stereo views, pose timing and comfort constraints	Render from runtime-provided timing and coordinate contracts.
39.4 · Portability, platform services and console-development boundaries	Build transferable systems without assuming proprietary access.

Read the module-project brief in the workbook before beginning. The four lessons build evidence for that project rather than four unrelated demonstrations.

Lesson 39.1 — Input-to-display latency and frame pacing

Learning outcome: Account for the whole interaction path rather than only frame rate.

A frame rate does not identify an input's journey

An input can wait for sampling, event delivery, a simulation step, rendering, a submission queue, presentation, scanout and the display's response. Average FPS compresses this journey into a throughput statistic. A system may complete 60 frames every second while presenting work based on older input. Conversely, a brief frame-time spike can make an otherwise high-throughput system feel inconsistent.

Define timestamp boundaries such as input generated, input consumed, simulation state committed, render submitted and presentation callback. Distinguish a software presentation marker from the first changed photon. OS events and engine markers can use different clocks or refer to different stages. SDL's event structures provide timestamps, but their presence does not turn an engine log into an optical measurement. [S13] A real photon measurement needs a suitable physical method, calibration and stated display position; the core lab does not require one.

Worked example: queue depth adds old work

At 60 Hz, one refresh interval is 1000/60 = 16.6667 ms. **Two intervals are approximately 33.3333 ms**, the original lesson's calculation. If a newly rendered result must wait behind two whole intervals of queued work under a stated schedule, that waiting can dominate a fast input-processing function. This is a simplified scheduling example, not a universal formula for every swapchain or presentation mode.

Suppose an input is generated at 100 ms, consumed at 103 ms, simulation commits at 107 ms, rendering finishes at 114 ms and a synthetic presentation marker occurs at 133 ms. The measured input-to-marker latency is 33 ms. The stage intervals are 3, 4, 7 and 19 ms. None measures panel response. A timestamp called “present” must include its exact meaning; renaming it “photon” without a measurement is an evidence error.

Frame pacing is the distribution of intervals

For presentation markers at 0, 16, 32 and 64 ms, intervals are 16, 16 and 32 ms. Mean interval is 21.3333 ms and reciprocal-mean interval throughput is 46.875 FPS. The two smooth intervals do not erase the doubled interval. Averaging the three individual FPS values instead gives a different, usually misleading summary for total completed work per elapsed time.

Report enough of the interval distribution to expose stalls and bursts. Nearest-rank p95 is useful as a defined statistic but does not summarize every tail. Annotate meaningful events, such as shader compilation or resource recreation, without automatically excluding them. Android's frame-pacing guidance treats presentation timing as a pipeline concern rather than just drawing as fast as possible. [S14]

Optimize with a causal timeline

An extra buffered frame can improve resource utilization while increasing latency. Reducing queue depth can expose a CPU or GPU starvation problem. Test both steady throughput and latency under a representative workload; do not assume every queue should be zero. Fixed-step simulation adds its own waiting and interpolation behavior, which must match the engine's contract.

The supplied trace uses a single synthetic clock and a marker explicitly named `synthetic_present`. The analyzer rejects a claim of photon timing. Extend your full assignment with a detailed input-to-display budget, marking unmeasured stages as unknown or modeled. A defensible result may be "input-to-software-marker p95 improved; optical latency was not measured." That is more informative than an unsupported claim about total responsiveness.

Practical assignment and assigned study

Original assignment: Instrument input receipt, simulation consumption, render submission and presentation-related timestamps. Pass: explain which stages are actually measured and which remain estimated, and show a frame-pacing distribution.

Assigned reading: R22: input and event timing; R24 or R25: presentation; R52: frame timing as an advanced comparison.

Run `python -m labs.run demo 39.1` from the package root. Implement `labs/starters/lesson_39_1.py`, then run `python -m labs.run check 39.1 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Is 60 frames per second a complete latency measurement?
2. What is two 60 Hz frame intervals?
3. Why label timestamp boundaries precisely?
4. Which is correct for throughput: reciprocal of mean interval or mean of interval reciprocals?
5. A trace measures input to software presentation callback. Can it establish panel response time?

Lesson 39.2 — Mobile and handheld systems: memory, power and lifecycle

Learning outcome: Design for suspension, limited budgets and sustained use.

A platform may stop your process without completing your plan

A desktop-like assumption of unlimited memory, continuous foreground execution and a generous shutdown callback does not transfer to every platform. Treat volatile runtime state, reconstructible resources and durable progress separately. Checkpoint meaningful progress while execution is healthy, not only at the last possible callback. Android's lifecycle guidance documents lifecycle transitions and process-related limits; a teaching state machine is not a substitute for the selected platform's persistence contract. [S15]

A useful engine state model includes RUNNING, SUSPENDED and KILLED. Suspension pauses relevant work and releases or marks resources according to policy. A kill loses volatile state. Launch reconstructs resources and loads a valid durable checkpoint. Rendering handles from an old resource generation must not be used after recreation, even if a numeric handle happens to be reused.

Worked example: a checkpoint has a specific promise

Start at game tick 0. Advance to tick 10 and checkpoint. Advance to tick 17 and simulate sudden process death. Relaunch restores tick 10, not 17. The seven uncheckpointed ticks are lost under this declared model. A graceful suspension might checkpoint tick 17, but an unexpected kill cannot depend on that callback. The exercise's suspend event deliberately performs a checkpoint; this is an explicit teaching policy rather than a universal operating-system guarantee.

Durable storage requires more than copying an object into a Python variable. A production design needs a versioned format, validation, an interruption-safe update strategy supported by the storage system, and a recovery policy for a partial or corrupt save. The core model represents durability symbolically. The full project should test corruption and interrupted writes in a disposable file-backed prototype rather than asserting production durability from the symbolic model.

Peak memory often occurs during a transition

Suppose a constrained renderer has a 256 MiB budget, 120 MiB of persistent engine data and an 80 MiB render target set. Steady use is 200 MiB. Replacing that set with another 80 MiB allocation before releasing the first raises the peak to 280 MiB, exceeding budget by 24 MiB. A steady-state memory screenshot misses the failure. Include staging buffers, decompression, alignment and old/new resource overlap in a peak budget.

Quality tiers should shed optional visual cost while preserving gameplay correctness and supported accessibility behavior. Reduce resolution, optional effects or cosmetic density under a stated policy; do not silently drop authoritative unit updates in an RTS. Tier transitions themselves need memory and time budgets. Use hysteresis and test repeated oscillation to reveal leaks or repeated allocations. Sustained performance also depends on power and thermal management, as explored in Lesson 38.4. [S12]

Make scarcity part of testing

A strict platform simulation should deny allocations above its budget, revoke resource generations after a lifecycle event and reject forbidden work while suspended. Silent success is a poor mock because it lets desktop assumptions survive. Test save-before-kill, kill-before-save, repeated suspend/resume, an allocation failure during recreation and a malformed saved version.

The supplied slice models progress, checkpointing and generations; it does not implement a filesystem or a real operating-system lifecycle. Completing the original assignment also requires a restricted-memory or lifecycle backend and a quality-tier decision with evidence. The model provides a repeatable way to develop these contracts before involving an actual handheld.

Practical assignment and assigned study

Original assignment: Add suspend/resume simulation and a constrained memory mode. Pass: progress survives the documented lifecycle, resources are restored safely and quality changes do not alter authoritative simulation rules.

Assigned reading: R25: platform graphics resources; R22: application lifecycle events; R29: text and input usability; R51: power-management concepts.

Run `python -m labs.run demo 39.2` from the package root. Implement `labs/starters/lesson_39_2.py`, then run `python -m labs.run check 39.2 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why not assume shutdown always provides unlimited save time?
2. Why measure sustained rather than only initial performance?
3. What should rendering quality tiers preserve?
4. After checkpointing tick 10, advancing to tick 17 and receiving a sudden kill, what does the model restore?
5. Why attach a generation to recreated rendering resources?

Lesson 39.3 — XR spaces, stereo views, pose timing and comfort constraints

Learning outcome: Render from runtime-provided timing and coordinate contracts.

A pose describes a transform at a time

A position and orientation need units, a reference space, a handedness convention and a timestamp. A headset pose relative to a local tracking origin is not automatically a world-space camera. Compose the tracking-to-world transform with the runtime pose, and keep the inverse transformation used by rendering distinct from the pose itself.

For a rigid camera transform with rotation R and translation p, the world-to-camera view transform uses `R^T` and `-R^T p`, not simply a negated position independent of rotation. Write a small test with a known point directly ahead of the camera. A scene that looks plausible from one orientation can conceal a sign error that becomes severe after a turn.

Frame timing and view data belong together

OpenXR's frame timing and view-location functions provide a public contract for requesting view data at a selected display time. Use the runtime's supported spaces, validity flags and per-view information instead of assuming every headset shares the same projection. The assigned references describe those API contracts; the core model does not initialize an XR runtime. [S16; S17]

Prediction estimates future motion; it is not a time machine. A constant-velocity prediction `p_future = p_now + v × delta_t` assumes velocity remains constant during the interval. With v=1 m/s and delta_t=0.02 s, the predicted translation is 0.02 m. Acceleration, tracking uncertainty and scheduling changes introduce error. A more elaborate model should be validated against representative motion and the relevant time boundary, not accepted because the scene appears smooth in one trial.

Worked example: stereo eye offsets rotate with the head

The teaching model uses meters, +Y up and -Z forward, with right-handed yaw rotation about +Y. For interpupillary distance 0.064 m, local eye offsets are -0.032 m and +0.032 m along local X. At zero yaw, these become world X offsets. At +90 degrees yaw under the stated matrix, the left eye offset becomes +0.032 m along Z and the right eye offset becomes -0.032 m along Z. The eyes rotate with the head; applying offsets permanently in world X is incorrect.

The model predicts translation, rotates eye offsets and preserves the chosen yaw over its bounded prediction interval. It does not predict angular velocity, provide a display distortion model or generate runtime-specific asymmetric projection matrices. The full assignment adds two camera views and validates the world-to-view transforms. A desktop diagram or numerical test can satisfy that core learning goal without a headset.

Tracking loss is a state transition, not new data

When tracking is invalid, preserve the application's defined recovery behavior and do not fabricate a fresh precise pose. The supplied model returns an explicit tracking-lost result. A real application may use runtime guidance and a conservative visual policy, but it must distinguish valid location, valid orientation, stale data and unsupported space relationships. Test origin changes and repeated loss/recovery so the world does not jump because two spaces were confused.

Latency and inconsistent motion cues matter to experience, but a passing transform test cannot establish comfort or safety for every person. Keep physical headset experiments optional, follow device guidance, and stop a session that causes discomfort. Report mathematical correctness and timing evidence separately from subjective or clinical claims. Stereo correctness, prediction validity and end-to-end responsiveness are related questions, not interchangeable evidence.

Practical assignment and assigned study

Original assignment: Implement two simulated camera views and a pose-time log, or a minimal OpenXR scene on available supported hardware. Pass: spaces and units are explicit, frame timing follows the runtime and loss of tracking has defined behavior.

Assigned reading: R52: spaces, view configuration, frame loop and timing; R29: interaction and accessibility review.

Run `python -m labs.run demo 39.3` from the package root. Implement `labs/starters/lesson_39_3.py`, then run `python -m labs.run check 39.3 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why is a pose incomplete without a space and time?
2. Should stereo projection be hard-coded for every headset?
3. Does prediction remove all latency or discomfort risk?
4. Why is the view translation -R^T p rather than simply -p?
5. At 1 m/s over a 20 ms prediction interval, what translation is predicted?

Lesson 39.4 — Portability, platform services and console-development boundaries

Learning outcome: Build transferable systems without assuming proprietary access.

Portability is a contract with failure cases

A platform abstraction should describe what the engine requires and how unsupported behavior is reported. An interface whose every mock call succeeds merely hides platform assumptions. Test required capabilities, optional capabilities, storage permissions, resource loss, user identity changes and unavailable services with explicit outcomes.

Separate minimum requirements from optional enhancements. For a hypothetical engine requiring `render` and `save`, a backend lacking `save` should fail initialization under that contract rather than begin a session that cannot preserve progress. A missing optional `haptics` capability can disable the effect while preserving the game. A “fallback” must actually implement the promised behavior; replacing a failed save with a success return is data loss, not portability.

Packaged assets and persistent data have different lifetimes

Treat installed assets as read-only unless the selected platform explicitly documents otherwise. Resolve save data through the platform's writable persistent-storage interface, not through a relative path that happens to work on a developer machine. Normalize and validate caller-supplied names under the intended storage root, and test errors such as a full quota, denied access, interrupted writes and unavailable user context. The mock should be restrictive enough to reveal assumptions before a port begins.

A save format also needs versioning, validation and a recovery policy. Platform services may be asynchronous and may disappear during suspend or account changes. Tie requests to their context and lifetime rather than retaining a stale callback indefinitely. Lesson 40.3's request generations provide a useful analogy, though a platform service API has its own specific contract.

Worked example: capabilities, not device-name guesses

Backend A advertises `{render, save, haptics}`. Backend B advertises `{render, save}`. Backend C advertises `{render}`. Under required `{render, save}` and optional `{haptics, cloud_save}`, A starts with haptics and without cloud save; B starts without either optional feature; C fails the stated minimum contract. The supplied capability negotiator produces these outcomes. It does not certify that an advertised capability is correctly implemented; behavioral tests remain necessary.

Capabilities can change over time. A user may revoke a permission or a resource may become unavailable. Decide whether the engine retries, falls back, pauses the affected operation or exits safely. Log a machine-readable failure category without exposing private user data. A capability snapshot at startup is not a permanent guarantee.

Public study is not proprietary platform authorization

The original curriculum explicitly separates public engine study from proprietary console development. This package contains no console SDK, private documentation, development credentials or certification claim. Actual platform access, distribution and certification depend on the platform holder's current authorized processes and agreements. Use official documentation available to your project and do not infer access from this course or a public API overview. [Original Volume 08, Lesson 39.4]

The core exercise is a strict backend simulation, which can run on an ordinary computer. It should test read-only assets, constrained saves, unsupported features and lifecycle loss. The supplied slice covers capability negotiation only; the full assignment requires the stricter platform behavior. Keep platform-specific details behind a narrow implementation boundary while keeping platform constraints visible in tests and product requirements. Portability fails when the abstraction erases a constraint that the application must actually handle.

Practical assignment and assigned study

Original assignment: Write a platform interface and capability matrix, then implement a second backend or strict mock. Pass: assumptions about paths, input and lifecycle are exposed, and proprietary requirements are marked unavailable until authorized.

Assigned reading: R22, R24 and R25: platform interfaces; R29: cross-device usability; official authorized platform documentation only for restricted targets.

Run `python -m labs.run demo 39.4` from the package root. Implement `labs/starters/lesson_39_4.py`, then run `python -m labs.run check 39.4 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why can a strict mock help portability?
2. Does public self-study grant access to proprietary console SDKs?
3. Why separate packaged assets from persistent saves?
4. What should happen when a required capability is missing?
5. Does successful capability negotiation prove the backend is correct?

Module 40 — Hardware/software co-design capstone

Prerequisites: Gates F and G; Modules 36-39.

Original module project: Gate H: choose one game workload and compare a software baseline with a specialized hardware or architecture alternative. Deliver a reference implementation, model or RTL, software interface, verification evidence and an end-to-end tradeoff report.

Lesson	Learning outcome
40.1 · Workload characterization and measurable requirements	Derive hardware goals from a real software operation.
40.2 · Architecture alternatives, arithmetic precision and dataflow	Explore several designs before committing to one.
40.3 · Interface design, virtualization and end-to-end evaluation	Include the host software in the hardware result.
40.4 · Design review, evidence tiers and undergraduate CE completion	Defend a co-design decision with limitations and reproducible artifacts.

Read the module-project brief in the workbook before beginning. The four lessons build evidence for that project rather than four unrelated demonstrations.

Lesson 40.1 — Workload characterization and measurable requirements

Learning outcome: Derive hardware goals from a real software operation.

Choose the workload before choosing the accelerator

A hardware proposal needs a measured reason to exist. Capture the engine workload's operation counts, batch sizes, data distributions, frame contribution, memory movement and numerical-quality requirements. Record what happens in a small scene, a typical scene, a large scene and an adverse case. A synthetic benchmark can explore a mechanism, but it does not establish how often a real game needs that mechanism.

A requirement must identify a metric, unit, boundary, workload and acceptance method. “Fast” becomes something like “under the declared scene and quality settings, the added operation must not increase p95 input-to-software-marker latency beyond the agreed budget.” The budget value is a project decision, not a number supplied by this course. Separate hard correctness and ownership requirements from performance goals that may be traded against cost or rejected after analysis.

Worked example: the original 0.2 ms opportunity

The original lesson considers a **0.2 ms operation in a 20 ms frame**. Removing it entirely saves at most 0.2/20 = **1% of frame time** under an additive critical-path model. Remaining frame time is 19.8 ms and maximum speedup is 20/19.8 ≈ 1.0101. A 1% time reduction and a 1.0101x speedup are related but not numerically identical percentages.

If setup and transfer add 0.25 ms, even an instantaneous compute unit makes that frame slower. If the operation was already hidden behind a longer independent task, removing it may save no critical-path time. These calculations can reject an architecture before expensive implementation. A rigorous negative result is a successful outcome when it answers the original engineering question.

Define a competent baseline

The baseline should use a suitable algorithm, an appropriate language and compiler configuration for the target, reasonable data layout, correct batching and the same quality requirement as the alternative. Do not compare optimized modeled hardware with an intentionally inefficient interpreted loop and call the ratio an application speedup. A Python oracle can establish expected outputs without being the performance baseline.

The supplied capstone includes native C++ contiguous and blocked candidates for a small exact affine workload. They provide a runnable benchmark example, not proof of the best engine implementation. A learner must connect the workload to actual engine evidence, assess a relevant CPU or existing GPU path, and explain why the selected baseline is competent. For a supported analytical route, combine measured software evidence with explicitly assumed architecture parameters rather than presenting both as physical measurements.

Build a requirement-to-evidence chain

Assign stable IDs to requirements, datasets, implementations, tests and claims. A quality claim should point to the exact oracle and adversarial inputs. A latency claim should point to raw timing boundaries and summaries. A resource claim should point to synthesis evidence when available or be labeled a bounded structural estimate when it is only a model. Hashes help readers identify files; they do not make a result correct.

Finally, list falsifiers: what observation would make you withdraw the proposal? Examples include transfer overhead larger than the available saving, unacceptable saturated outputs, a missed tail case, or a memory budget violation. State the decision rule before selecting the most flattering workload. Your capstone should explain both why one design was selected and why credible alternatives were not.

Practical assignment and assigned study

Original assignment: Select a workload from the engine and write a requirement sheet plus baseline measurements. Pass: each target has units and a test, and the maximum plausible application benefit is estimated.

Assigned reading: R43: bottleneck modeling; R48: quantitative design; R49: project methodology.

Run `python -m labs.run demo 40.1` from the package root. Implement `labs/starters/lesson_40_1.py`, then run `python -m labs.run check 40.1 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why start with workload measurements?
2. What maximum fraction of the example frame can removing 0.2 ms save?
3. Can rejecting a hardware proposal be a successful outcome?
4. Why is a Python correctness oracle not automatically a competent performance baseline?
5. What should a performance requirement specify besides a number?

Lesson 40.2 — Architecture alternatives, arithmetic precision and dataflow

Learning outcome: Explore several designs before committing to one.

Architecture is the arrangement of work and movement

Compare at least two credible alternatives beyond the competent baseline. A scalar pipeline, a multi-lane pipeline and an existing GPU path have different setup costs, concurrency, memory demands and verification burdens. Draw where each value resides, when it moves, who owns it and how backpressure is handled. More arithmetic lanes do not create more memory bandwidth.

In the supplied model, each item consumes a signed 16-bit input and produces a signed 16-bit output: four bytes of total modeled memory traffic. With 400 MB/s of internal bandwidth, memory can serve at most 100 million items/s under this ideal traffic assumption. One item per cycle at a hypothetical 100 MHz reaches the same asymptotic item rate. Increasing arithmetic lanes above one cannot exceed that memory ceiling. The clock and bandwidth are assumptions, not synthesized or measured properties.

Latency and initiation interval are different

A pipeline may accept work every cycle while each item spends several cycles inside it. For a no-stall model with G accepted groups, latency L cycles from first acceptance to first retirement, and initiation interval 1, the interval from first acceptance to last retirement is `L + G − 1`. The supplied simulator names this `elapsed_cycles`. It also records `tick_count`, which includes both sampled boundary ticks and is one larger for a nonempty no-stall run. State the edge convention before converting counts to seconds.

The teaching pipeline evaluates the arithmetic at acceptance and moves packets through delay registers. It models order, grouping, tails and stalls; it is not RTL synthesis or proof that arithmetic meets a clock target. Global stalling freezes the modeled stages when an occupied output cannot retire. A more realistic elastic pipeline is an extension with a different flow-control contract.

Worked example: define fixed-point arithmetic exactly

A signed Q8.8 value stores an integer representing a real number divided by 256. Its raw signed-16 range is -32768 to 32767, corresponding to -128 to 127.99609375. The capstone operation multiplies by encoded gain, rounds the scaled product to nearest with ties to even, adds encoded bias, then saturates to the signed range. The order matters: rounding after adding a fractional bias is a different operation.

For a gain of 128 (0.5) and zero bias, raw inputs 1, 3, -1 and -3 produce raw outputs 0, 2, 0 and -2 with nearest-even rounding. This tests both tie parity and negative symmetry. A raw value 3 represents 3/256, so do not confuse raw-code error with error in real units. Saturation prevents wraparound but can create unacceptable clipping; count saturated results separately from ordinary rounding error.

Error is a requirement, not a cosmetic detail

Choose precision from the application's allowed error and dynamic range. For a visual filter, an image-quality criterion may be appropriate; for authoritative simulation, deterministic state and threshold decisions may require stricter semantics. Test extremes, values near rounding ties, values near saturation and sequences that amplify error. A small average error can hide a catastrophic boundary case.

The full design comparison should include useful throughput, latency, memory, structural resource estimates, setup/transfer and quality. Resource estimates such as “four multiplier instances” are not LUT counts, power estimates or ASIC area. The original program's co-design and simulation references support progressively deeper investigation, but each result must retain its evidence tier. [S18; S19; R49]

Practical assignment and assigned study

Original assignment: Model three variants, such as scalar, pipelined and multi-lane. Pass: resource and bandwidth estimates are explicit, fixed-point error meets a chosen application tolerance and rejected variants have documented reasons.

Assigned reading: R49: architecture and RTL modeling; R43: bandwidth limits; R33: resource inference.

Run `python -m labs.run demo 40.2` from the package root. Implement `labs/starters/lesson_40_2.py`, then run `python -m labs.run check 40.2 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why might twice as many lanes fail to double throughput?
2. What should determine acceptable quantization error?
3. Why explore more than one architecture?
4. What is the maximum representable signed Q8.8 value?
5. What does the pipeline model’s clock assumption prove about physical timing?

Lesson 40.3 — Interface design, virtualization and end-to-end evaluation

Learning outcome: Include the host software in the hardware result.

The interface is part of the accelerator

A work descriptor identifies operation, buffer, offset, element count, format, request ID and execution context. Validate it before allowing access. Bounds checks must resist arithmetic overflow in a fixed-width implementation: after checking `offset <= size`, compare `count <= (size-offset)/stride` rather than trusting an unchecked `offset + count*stride`. Also validate alignment, allowed operation, registered memory, ownership and the declared numerical format.

The supplied descriptor exercise uses four-byte elements in its standalone buffer model. The affine capstone uses two-byte input and two-byte output samples. These are deliberately separate teaching contracts; a real integrated driver must define its actual buffer strides and adapt validation accordingly. Do not connect two independently correct modules while leaving this disagreement implicit.

Ownership survives an inconvenient timeout

The educational lifecycle is CPU_OWNED, DEVICE_OWNED and QUARANTINED. Submission transfers ownership. A valid completion may return ownership under the declared policy. Timeout quarantines the buffer because the device might still touch it. A late completion received while quarantined is ignored in this model; reuse requires a reset operation representing acknowledged quiescence. Incrementing an epoch alone does not stop real hardware. The model's reset explicitly assumes that stopping condition has already been established.

A completion carries the request identity and epoch. After a quiescent reset, old completions cannot satisfy a new request even when numeric slots are reused. Test duplicate completion, stale epoch, completion after timeout, removal during work and attempted CPU access while device-owned. Linux's DMA guidance is an assigned reference for real buffer mapping and ownership concerns; the Python model performs no DMA. [S05]

Isolation is more than bounds within one array

Multiple users or virtual machines add contexts, address translation, scheduling fairness and fault containment. A descriptor valid in one context must not authorize access to another context's memory. An IOMMU or a supported device virtualization mechanism can participate in isolation, but mentioning one in a diagram does not implement the policy. The core exercise uses a single registered buffer and makes no multi-tenant security claim.

Define how reset affects other requests and whether a device-wide reset interrupts unrelated users. An interface contract should state resource limits, admission policy, completion ordering, cancellation semantics and which failures force a fallback. These choices affect throughput and tail latency even when arithmetic is unchanged.

Worked example: setup creates a crossover

Assume a competent software path takes 0.20 microseconds per item. A modeled accelerator takes 20 microseconds setup plus 0.05 microseconds per item, with all transfer and synchronization already included in those stated terms. The accelerator wins only when `20 + 0.05N < 0.20N`, or `N > 133.333...`; the first integer winning size is 134. Changing the transfer assumption moves the threshold. A model that omits transfers cannot establish this end-to-end crossover.

Kernel and application speedups can legitimately differ. An unchanged frame portion, result readback, format conversion, queue wait and failure handling reduce or erase benefit. Report input preparation through useful result consumption, plus separate phase timings. Test the batch sizes the game actually produces, not only the size that fills the proposed machine. The original assignment's end-to-end requirement is central to Gate H, not an optional final paragraph.

Practical assignment and assigned study

Original assignment: Implement a host driver model with invalid-descriptor, timeout and reset tests. Pass: buffer bounds are enforced, ownership survives failure and results include both kernel and end-to-end timing.

Assigned reading: R51: DMA and device interfaces; R49: system integration; R38: modeled systems when hardware is unavailable.

Run `python -m labs.run demo 40.3` from the package root. Implement `labs/starters/lesson_40_3.py`, then run `python -m labs.run check 40.3 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why validate device work descriptors?
2. Can kernel and application speedups legitimately differ?
3. Why test realistic batch sizes?
4. Does increasing an epoch make a timed-out DMA buffer physically safe to reuse?
5. Why prefer count <= (size-offset)/stride after validating offset?

Lesson 40.4 — Design review, evidence tiers and undergraduate CE completion

Learning outcome: Defend a co-design decision with limitations and reproducible artifacts.

A design review asks what the evidence establishes

A functional reference establishes expected behavior under a stated contract. A functional model checks an implementation of that behavior. A cycle model adds a declared timing abstraction. Synthesis produces tool- and target-specific structural evidence. A host measurement records behavior of executed host software in its environment. Physical measurement observes a built device through a method with its own limits. These tiers answer different questions and should not be blended into one adjective such as "validated."

The supplied package includes functional and declared-cycle models, analytical cost estimates and optional native host measurements. It does not include a fabricated board, synthesized accelerator, timing closure or measured GPU/XR behavior. The evidence ledger labels those distinctions. A JSON auditor can verify referenced files, hashes and required fields; it cannot decide whether a baseline is fair or a scientific conclusion follows.

Worked example: a correct result with an unsupported clock

A model produces exact results for all tested batches and assumes a 100 MHz device. The functional evidence supports those tested results. The assumed frequency supports a conditional timing estimate: "at the assumed clock and modeled stalls, this case takes X seconds." It does not support "the hardware runs at 100 MHz." To make that stronger claim, a suitable implementation, timing evidence and possibly physical measurement would be needed at the appropriate tier.

Likewise, "four multipliers" is a structural model count, not silicon area or board power. A successful host benchmark establishes the observed software timing boundary in its recorded environment, not the speed of the modeled architecture. The report should let a reader trace each number to the experiment or assumption that produced it.

Present the strongest competing explanation

An optimization may appear faster because the baseline used a different quality level, a warm cache, a smaller input, a different compiler option or a timing boundary that omitted completion. Challenge these possibilities yourself. Include the negative cases: tiny batches, tails, saturation boundaries, reduced bandwidth, delayed completion and constrained memory. A reviewer should not need to guess which condition makes the proposal fail.

Reproducibility means another person can recover the workload, build or run the implementation, execute tests and regenerate the reported summaries. Preserve version metadata, seeds, raw samples, commands and limitations. Do not edit raw data to match a model. A reproducible result can still be wrong, but it is much easier to examine and correct.

Gate H closes the undergraduate engineering track

The original Gate H requires a measured engine workload, a competent baseline, at least two architecture alternatives, and one implemented specialized model or hardware path with clear limits. The required portfolio includes workload distributions, numerical-quality limits, ownership, setup/transfer costs, resource/timing estimates, failure handling and sensitivity analysis. A negative outcome is acceptable when the reasoning and evidence are rigorous. [Original program Gate H]

The supplied teaching capstone shows how to assemble such evidence, but it is not a completed personal Gate H submission: its affine workload is synthetic, its architecture parameters are assumed, and its standalone ownership exercise requires adaptation to an integrated project. Replace those assumptions with your project's evidence or retain them as explicit conditional estimates. Have another person challenge the requirement-to-evidence chain.

Passing this self-study checkpoint demonstrates the submitted learning outcomes under review; it does not confer an accredited degree, professional license, console authorization or production-fabrication approval. The next volume deepens graduate game-engine and graphics engineering. Carry forward the discipline developed here: state the contract, build the smallest testable model, measure a relevant boundary, and keep every claim no stronger than its evidence.

Practical assignment and assigned study

Original assignment: Complete Gate H and a design-review presentation. Pass: every major claim traces to an artifact, the baseline is reproduced and a limitations section clearly separates educational estimates from production claims.

Assigned reading: R49: design-project reporting; R48: architectural tradeoffs; revisit the evidence and project rubrics in the handbook.

Run `python -m labs.run demo 40.4` from the package root. Implement `labs/starters/lesson_40_4.py`, then run `python -m labs.run check 40.4 --student`. Use the workbook for the contract, full lab steps, evidence, written problems and transfer task. Passing the slice alone does not complete the original assignment.

Check your understanding

Attempt all five questions before opening the answer key.

1. Why label simulation and physical measurements separately?
2. What makes a baseline competent?
3. Does Gate H authorize production chip fabrication?
4. Does a matching file hash prove an experiment is scientifically sound?
5. May a well-supported negative result pass Gate H?

Sources and provenance

How references are used

The original curriculum's R-codes are retained for assigned study. S-codes identify selected official documentation and primary references checked while preparing this expansion on September 28, 2026. Moving documentation can change; use the version appropriate to the installed tool or target. A versioned KiCad 9.0 link is a deliberate teaching reference, not a claim about the latest release. No third-party manuals or proprietary SDKs are redistributed.

Original lesson IDs, titles, outcomes, practical briefs, assigned readings, module projects and first three quiz questions come from the supplied Game_Engineering_Program_Manifest.json and COMPLETE_CURRICULUM.pdf. The extracted Volume 08 manifest, original quiz text and complete original R-code directory are retained in source/ORIGINAL_VOLUME_08_MANIFEST.json. New explanations, worked examples, code, exercises, added questions and synthetic traces are original teaching material in this expansion. They do not imply accreditation or independent peer review.

Numerical examples labeled hypothetical or synthetic are not product specifications. The native reference measurements were actually executed on the build host; their method and limitations are in capstone/results/environment.json. No board, GPU accelerator, headset, KiCad GUI or proprietary console SDK was executed as part of this package's automated validation.

Selected official and primary sources

S01 — Zephyr Project: SPI peripheral API. Role, mode and transaction vocabulary. Use the exact device datasheet for electrical limits.

https://docs.zephyrproject.org/latest/hardware/peripherals/spi.html

S02 — Zephyr Project: Scheduling. RTOS scheduling mechanisms; the supplied integer-tick scheduler is an original simplified model.

https://docs.zephyrproject.org/latest/kernel/services/scheduling/index.html

S03 — KiCad Project: Schematic Editor, version 9.0. Schematic editing and electrical-rule checking. Versioned documentation, not a claim that 9.0 is the latest release.

https://docs.kicad.org/9.0/en/eeschema/eeschema.html

S04 — KiCad Project: PCB Editor, version 9.0. Board layout and design-rule checks. Use documentation matching the installed version.

https://docs.kicad.org/9.0/en/pcbnew/pcbnew.html

S05 — Linux kernel contributors: Dynamic DMA mapping guide. Mapping, access and ownership considerations. The educational Python code does not use DMA.

https://docs.kernel.org/core-api/dma-api-howto.html

S06 — NVIDIA: CUDA Programming Guide: Writing SIMT Kernels. GPU execution-group vocabulary; architecture parameters must be checked for the selected target.

https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html

S07 — NVIDIA: CUDA C++ Best Practices Guide. Performance, memory access and validation context. No CUDA runtime is required for the core labs.

https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

S08 — NVIDIA: CUDA Programming Guide: Unified and System Memory. Separate addressing, placement and access behavior. Actual support depends on the platform.

https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/understanding-memory.html

S09 — Khronos Group: Vulkan Guide: Synchronization. Execution and memory synchronization concepts; consult the selected API specification for implementation.

https://docs.vulkan.org/guide/latest/synchronization.html

S10 — Samuel Williams, Andrew Waterman and David Patterson / UC Berkeley: Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures, technical report EECS-2008-134. Original performance-model reference. All numerical worksheet examples in this package are original hypothetical values.

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

S11 — Python Software Foundation: timeit — Measure execution time of small code snippets. Timing boundaries and repeated experiments. Package execution was tested on Python 3.13.5, not every documented version.

https://docs.python.org/3/library/timeit.html

S12 — Android Developers: Android Dynamic Performance Framework. Device-dependent thermal and performance-management context; physical use remains optional.

https://developer.android.com/games/optimize/adpf

S13 — SDL Project: SDL3: SDL_Event. Event representation and timestamps. An event timestamp is not a photon measurement.

https://wiki.libsdl.org/SDL3/SDL_Event

S14 — Android Developers: Frame Pacing library. Presentation and frame-pacing context. The supplied trace is synthetic, not captured by this library.

https://developer.android.com/games/sdk/frame-pacing

S15 — Android Developers: The activity lifecycle. Lifecycle and state-management context. The teaching state machine intentionally simplifies operating-system behavior.

https://developer.android.com/guide/components/activities/activity-lifecycle

S16 — Khronos Group: OpenXR 1.1 reference: xrWaitFrame. Frame timing contract. No real XR runtime was executed for this package.

https://registry.khronos.org/OpenXR/specs/1.1/man/html/xrWaitFrame.html

S17 — Khronos Group: OpenXR 1.1 reference: xrLocateViews. Space-, time- and view-dependent pose information.

https://registry.khronos.org/OpenXR/specs/1.1/man/html/xrLocateViews.html

S18 — Cornell University: ECE 5745: Complex Digital ASIC Design. Deeper digital-design and evaluation study. This package is not a substitute for an ASIC implementation flow.

https://www.csl.cornell.edu/courses/ece5745/

S19 — gem5 Project: gem5 documentation. Supported architectural simulation as an optional deeper path; gem5 is not a required dependency.

https://www.gem5.org/documentation/

S21 — Apple Developer: Synchronizing CPU and GPU work. A platform-specific synchronization reference; the documentation page may require JavaScript.

https://developer.apple.com/documentation/metal/synchronizing-cpu-and-gpu-work

Original assigned-reading directory

The following R-codes are used by this volume or its review prerequisites. Their wording and URLs are retained from the original program directory; inclusion is an assigned-study pointer, not a claim that every linked external resource was reread in full for this edition. The full original directory remains in the JSON source.

R09 — MIT OpenCourseWare: 18.05 Introduction to Probability and Statistics. Probability, inference and quantitative reasoning.

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

R11 — C++ Core Guidelines editors: C++ Core Guidelines. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R21 — Khronos Group: Vulkan tutorial. Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22 — SDL project: SDL3 documentation. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23 — Matt Pharr, Wenzel Jakob, Greg Humphreys: Physically Based Rendering, fourth edition. Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24 — Microsoft: Direct3D 12 graphics. Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25 — Apple: Metal developer resources. Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26 — Epic Games: Programming with C++ in Unreal Engine. Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R29 — Microsoft: Xbox Accessibility Guidelines. Accessibility design and testing; not a legal-compliance certificate.

https://learn.microsoft.com/en-us/xbox/accessibility/guidelines

R30 — MIT OpenCourseWare: 6.002 Circuits and Electronics. Circuits, transistors and laboratory reasoning.

https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R33 — YosysHQ: Yosys documentation. Synthesis and technology mapping; stable installed release preferred over a development snapshot.

https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R36 — KiCad project: KiCad documentation. Schematic capture, PCB layout and manufacturing outputs.

https://docs.kicad.org/

R37 — NVIDIA: CUDA Programming Guide. GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

R38 — gem5 project: gem5 documentation. Architecture simulation; distinguish model outputs from measured hardware results.

https://www.gem5.org/documentation/

R43 — Samuel Williams, Andrew Waterman and David Patterson: Roofline performance model, technical report. Original 2008 performance-model reference; use current measured hardware limits for experiments.

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

R48 — MIT OpenCourseWare: 6.823 Computer System Architecture. Graduate architecture foundations; historical course concepts supplemented with current specifications.

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

R49 — Christopher Batten / Cornell University: ECE5745 Complex Digital ASIC Design. Graduate ASIC methodology, circuits, EDA and design-project expectations.

https://www.csl.cornell.edu/courses/ece5745/

R51 — Linux kernel contributors: Driver implementer API guide. Device ownership, I/O, DMA and driver interfaces; use disposable VMs or simulated devices for kernel experiments.

https://docs.kernel.org/driver-api/index.html

R52 — Khronos Group: OpenXR registry and specifications. XR coordinate spaces, frame timing and runtime interfaces; optional hardware-dependent extension.

https://registry.khronos.org/OpenXR/

R53 — Zephyr Project: Zephyr documentation. Embedded kernel, drivers, board support and simulation; select and pin a supported release.

https://docs.zephyrproject.org/latest/introduction/index.html